

✓ Lab exercise PT-P2 (Neural Network Training)

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remc

- `from google.colab import drive`: Imports the `drive` module from the `google.colab` package, which provides Python utilities specifically designed for Google Colab notebooks to interact with Google Drive.
- `drive.mount('/content/drive')`: Mounts your personal Google Drive file storage onto the local file system of the Colab virtual machine at the specified path (`/content/drive`). When run, it prompts you to authenticate, granting the notebook read and write access to your Drive files as if they were local files.

```
import pandas as pd
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, GlobalAveragePooling1D
from tensorflow.keras.optimizers import Adam, SGD
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score, accuracy_score
from collections import Counter
import matplotlib.pyplot as plt

np.random.seed(42)
tf.random.set_seed(42)
```

- `import pandas as pd`: Imports the Pandas library under the alias `pd`, used for structured data manipulation, analysis, and working with DataFrames.
- `import numpy as np`: Imports NumPy under the alias `np`, providing support for large, multi-dimensional arrays and high-level mathematical operations.
- `import tensorflow as tf`: Imports the main TensorFlow library as `tf` to construct, train, and run deep learning models.

- `from tensorflow.keras.models import Sequential`: Imports the `Sequential` model class, which allows you to build a neural network layer-by-layer in a linear stack.
 - `from tensorflow.keras.layers import Embedding, GlobalAveragePooling1D, Dense, Dropout`: Imports specific Keras neural network layers:
 - `Embedding`: Turns positive integers (word indexes) into dense vectors of fixed size (used for text processing).
 - `GlobalAveragePooling1D`: Calculates the average across the sequence length, flattening 3D tensor inputs to 2D.
 - `Dense`: A standard fully connected neural network layer.
 - `Dropout`: Randomly deactivates input units during training at a set rate to help prevent overfitting.
 - `from tensorflow.keras.optimizers import Adam, SGD`: Imports the Adam (Adaptive Moment Estimation) and SGD (Stochastic Gradient Descent) algorithms used to update model weights and minimize loss during training.
 - `from sklearn.model_selection import train_test_split`: Imports a helper function to split datasets into separate subsets for training and testing/validation.
 - `from sklearn.metrics import f1_score, accuracy_score`: Imports metric functions to calculate accuracy and F1-score for model performance evaluation.
 - `from collections import Counter`: Imports Python's built-in `Counter` class to quickly count frequency distributions of elements in lists or arrays (e.g., class distributions or vocabulary tokens).
 - `import matplotlib.pyplot as plt`: Imports Matplotlib's plotting interface under the alias `plt` for visualizing data, loss curves, and evaluation metrics.
 - `np.random.seed(42)`: Sets a fixed random seed for NumPy's pseudo-random number generator to ensure code reproducibility across runs.
 - `tf.random.set_seed(42)`: Sets a fixed random seed for TensorFlow so that weight initialization and operations like Dropout produce identical results each time the code is executed.
-
-

```

'''
EXP-01, 02, 03 & 04
CSV_FILEPATH = '/content/drive/MyDrive/NN Training/sample_dataset.csv'
'''

CSV_FILEPATH = '/content/drive/MyDrive/NN Training/modified_sample_
MAX_LEN = 20
MAX_VOCAB_SIZE = 1000

df = pd.read_csv(CSV_FILEPATH)

def build_vocab(texts, max_vocab_size):
    words = [word.lower() for text in texts for word in str(text).split()]
    word_counts = Counter(words)

    most_common = word_counts.most_common(max_vocab_size - 2)
    vocab = {word: idx + 2 for idx, (word, _) in enumerate(most_common)}
    vocab['<PAD>'] = 0
    vocab['<UNK>'] = 1
    return vocab

def text_to_sequence(text, vocab, max_len):
    tokens = str(text).lower().split()
    seq = [vocab.get(token, 1) for token in tokens]
    if len(seq) < max_len:
        seq += [0] * (max_len - len(seq))
    else:
        seq = seq[:max_len]
    return seq

vocab = build_vocab(df['text'].tolist(), MAX_VOCAB_SIZE)
encoded_texts = [text_to_sequence(txt, vocab, MAX_LEN) for txt in df['text'].tolist()]

X_data = np.array(encoded_texts)
y_data = np.array(df['label'].values)

num_classes = len(np.unique(y_data))

X_train, X_temp, y_train, y_temp = train_test_split(X_data, y_data,
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, tes

```

- `CSV_FILEPATH = '/content/drive/MyDrive/NN Training/sample_dataset.csv'`: Defines the string path leading to the input dataset stored on your mounted Google Drive.
- `MAX_LEN = 20`: Sets the maximum sequence length (number of words) for every input text sample.
- `MAX_VOCAB_SIZE = 1000`: Caps the vocabulary size at 1,000 unique tokens (including special control tokens).
- `df = pd.read_csv(CSV_FILEPATH)`: Reads the CSV file into a Pandas DataFrame stored in `df`.

- `def build_vocab(texts, max_vocab_size):`: Defines a function that creates a word-to-integer mapping dictionary from a list of text samples.
- `words = [word.lower() for text in texts for word in str(text).split()]`: Flattens all text samples into a single list of lowercased words by splitting strings on whitespace.
- `word_counts = Counter(words)`: Counts the occurrence frequency of every unique word across all input texts.
- `most_common = word_counts.most_common(max_vocab_size - 2)`: Extracts the top `N` most frequent words, leaving 2 reserved slots for special tokens.
- `vocab = {word: idx + 2 for idx, (word, _) in enumerate(most_common)}`: Constructs a dictionary mapping each common word to an integer index, starting at index 2.
- `vocab['<PAD>'] = 0`: Assigns index `0` to the padding token `<PAD>`, used to equalize sequence lengths.
- `vocab['<UNK>'] = 1`: Assigns index `1` to the unknown token `<UNK>`, reserved for words outside the top vocabulary.
- `return vocab`: Returns the completed vocabulary dictionary.
- `def text_to_sequence(text, vocab, max_len):`: Defines a function that converts a single text string into a fixed-length list of integer indices.
- `tokens = str(text).lower().split()`: Converts input text to string, lowercases it, and splits it into individual word tokens.
- `seq = [vocab.get(token, 1) for token in tokens]`: Maps each token to its integer index using `vocab`, defaulting to `1` (`<UNK>`) if the token is out-of-vocabulary.
- `if len(seq) < max_len:`: Checks if the sequence has fewer tokens than the allowed `MAX_LEN`.
- `seq += [0] * (max_len - len(seq))`: Appends `0` (`<PAD>`) to the right end of the sequence until its length equals `max_len`.
- `else:`: Executes if the sequence length is greater than or equal to `max_len`.
- `seq = seq[:max_len]`: Truncates the sequence to the first `max_len` tokens.
- `return seq`: Returns the final fixed-length integer sequence.

- `sequence = text_to_sequence(txt, vocab, MAX_LEN)`: Returns the maximum length integer sequence.
- `vocab = build_vocab(df['text'].tolist(), MAX_VOCAB_SIZE)`: Builds the vocabulary mapping dictionary from the `'text'` column of `df`.
- `encoded_texts = [text_to_sequence(txt, vocab, MAX_LEN) for txt in df['text']]`: Converts every text sample in `df['text']` into a padded/truncated integer sequence.
- `X_data = np.array(encoded_texts)`: Converts the list of integer sequences into a 2D NumPy array to serve as feature inputs.
- `y_data = np.array(df['label'].values)`: Extracts the target labels from `df` as a NumPy array.
- `num_classes = len(np.unique(y_data))`: Calculates the total number of distinct target class labels in the dataset.
- `X_train, X_temp, y_train, y_temp = train_test_split(X_data, y_data, test_size=0.30, random_state=42)`: Splits the dataset into 70% training data (`X_train`, `y_train`) and 30% temporary evaluation data (`X_temp`, `y_temp`).
- `X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.50, random_state=42)`: Splits the 30% temporary data equally, resulting in 15% validation data (`X_val`, `y_val`) and 15% testing data (`X_test`, `y_test`).

```
# EXP-05
CONFIG = {
    "run_id": "EXP-05",
    "embed_dim": 64,
    "hidden_dim": 128,
    "learning_rate": 0.001,
    "batch_size": 32,
    "epochs": 15,
    "optimizer_type": "Adam",
    "dropout_rate": 0.2
}

model = Sequential()

model.add(Embedding(
    input_dim=len(vocab),
    output_dim=CONFIG["embed_dim"],
    input_length=MAX_LEN,
    mask_zero=True
))
```

```
model.add(GlobalAveragePooling1D())

model.add(Dense(CONFIG["hidden_dim"], activation='relu'))

model.add(Dropout(CONFIG["dropout_rate"]))

model.add(Dense(num_classes, activation='softmax'))

...

# EXP-05A
CONFIG = {
    "run_id": "EXP-05A",
    "embed_dim": 64,
    "hidden_dim": 128,          # Defined here to prevent KeyError
    "learning_rate": 0.0005,
    "batch_size": 16,
    "epochs": 12,
    "optimizer_type": "Adam",
    "dropout_rate": 0.2
}

model = Sequential()

model.add(Embedding(
    input_dim=1000,
    output_dim=CONFIG["embed_dim"]
))

model.add(GlobalAveragePooling1D())

model.add(Dense(CONFIG["hidden_dim"], activation='relu'))

model.add(Dropout(CONFIG["dropout_rate"]))

model.add(Dense(3, activation='softmax'))
...

...

# EXP-05B
CONFIG = {
    "run_id": "EXP-05B",
    "embed_dim": 32,
    "hidden_dim": 64,
    "learning_rate": 0.01,
    "batch_size": 32,
    "epochs": 15,
    "optimizer_type": "SGD",
    "momentum": 0.9,
    "dropout_rate": 0.1,
    "max_vocab_size": 800,
    "max_len": 32
}
```

```

model = Sequential()

model.add(Embedding(
    input_dim=CONFIG["max_vocab_size"],
    output_dim=CONFIG["embed_dim"],
    mask_zero=True
))

model.add(GlobalAveragePooling1D())

model.add(Dense(CONFIG["hidden_dim"], activation='relu'))

model.add(Dropout(CONFIG["dropout_rate"]))

model.add(Dense(num_classes, activation='softmax'))

optimizer = tf.keras.optimizers.SGD(
    learning_rate=CONFIG["learning_rate"],
    momentum=CONFIG["momentum"]
)

model.compile(
    optimizer=optimizer,
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
...

```

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100:
warnings.warn(
'\n# EXP-05B\nCONFIG = {\n    "run_id": "EXP-05B",\n    "embed_dim":\n: 32,\n    "hidden_dim": 64,\n    "learning_rate": 0.01,\n    "batch_size": 32,\n    "epochs": 15,\n    "optimizer_type": "SGD",\n    "momentum": 0.9,\n    "dropout_rate": 0.1,\n    "max_vocab_size": 8000,\n    "max_len": 32\n}\n\nmodel = Sequential()\n\nmodel.add(Embedding(\n    input_dim=CONFIG["max_vocab_size"],\n    output_dim=CON

```

- `CONFIG = { ... }`: Defines a configuration dictionary to centralize all hyperparameter values, tracking identifiers, and training settings in one place.
 - `"run_id": "EXP-01-KERAS-BASELINE"`: A string tag used to identify and log this specific experiment run.
 - `"embed_dim": 32`: Sets the size (dimensionality) of the vector representation for each word in the embedding layer to 32.
 - `"hidden_dim": 64`: Sets the number of neurons/nodes in the fully connected hidden layer to 64.
 - `"learning_rate": 0.005`: Sets the step size the optimizer takes when updating model weights.
 - `"batch_size": 32`: Sets the number of training samples processed

- `(batch_size = 32)`: Sets the number of training samples processed together before updating model weights.
 - `"epochs": 15`: Sets the total number of complete passes through the training dataset during training.
 - `"optimizer_type": "Adam"`: Specifies the string name of the optimization algorithm to use.
 - `"dropout_rate": 0.1`: Sets the probability (10%) for randomly dropping network units during training to prevent overfitting.
 - `model = Sequential()`: Initializes an empty Keras `Sequential` model container, which allows you to stack neural network layers sequentially top-to-bottom.
 - `model.add(Embedding(...))`: Adds a lookup table layer that maps each integer-encoded word in an input sequence to a learnable dense vector.
 - `(input_dim=len(vocab))`: Tells the layer the total size of the vocabulary (number of unique row lookup entries).
 - `(output_dim=CONFIG["embed_dim"])`: Sets the output vector size for each word token to 32.
 - `(input_length=MAX_LEN)`: Explicitly defines the fixed input sequence length (20 tokens).
 - `(mask_zero=True)`: Instructs the model to ignore index `0` (`<PAD>`) in downstream computations so padded values don't affect learning.
 - `model.add(GlobalAveragePooling1D())`: Adds a pooling layer that computes the mean of the embedding vectors across the time/sequence length dimension, converting 3D sequence tensors into 2D feature representations.
 - `model.add(Dense(CONFIG["hidden_dim"], activation='relu'))`: Adds a fully connected layer with 64 neurons and a Rectified Linear Unit (`ReLU`) activation function to learn non-linear feature interactions.
 - `model.add(Dropout(CONFIG["dropout_rate"]))`: Adds a regularization layer that randomly zeros out 10% of the input activation units during each training step to prevent co-adaptation of features.
 - `model.add(Dense(num_classes, activation='softmax'))`: Adds the final output layer containing one neuron per target class. The `softmax` activation function converts the layer's raw output scores into a normalized probability distribution that sums to 1.0.
-

```

if CONFIG["optimizer_type"] == "Adam":
    opt = Adam(learning_rate=CONFIG["learning_rate"])
else:
    opt = SGD(learning_rate=CONFIG["learning_rate"])

model.compile(
    loss='sparse_categorical_crossentropy',
    optimizer=opt,
    metrics=['accuracy']
)

model.summary()

print(f"--- Starting Training Run: {CONFIG['run_id']} ---")

history = model.fit(
    X_train,
    y_train,
    validation_data=(X_val, y_val),
    epochs=CONFIG["epochs"],
    batch_size=CONFIG["batch_size"],
    verbose=1
)

val_preds_probs = model.predict(X_val)
val_preds = np.argmax(val_preds_probs, axis=1)

val_f1 = f1_score(y_val, val_preds, average='weighted')
val_loss = history.history['val_loss'][-1]

print("\n=== EXP LOG ENTRY ===")
print(f"ID: {CONFIG['run_id']} | LR: {CONFIG['learning_rate']} | Hi

```

Model: "sequential_16"

Layer (type)	Output Shape	F
embedding_16 (Embedding)	?	0 (ur
global_average_pooling1d_16 (GlobalAveragePooling1D)	?	
dense_30 (Dense)	?	0 (ur
dropout_15 (Dropout)	?	
dense_31 (Dense)	?	0 (ur

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

--- Starting Training Run: EXP-04 ---

Epoch 1/15

1/1  1s 1s/step - accuracy: 0.3000 - loss: 1.098

Epoch 2/15

```

Epoch 2/15
1/1 _____ 0s 84ms/step - accuracy: 0.6000 - loss: 1.6
Epoch 3/15
1/1 _____ 0s 81ms/step - accuracy: 0.7000 - loss: 1.6
Epoch 4/15
1/1 _____ 0s 100ms/step - accuracy: 0.7000 - loss: 1.6
Epoch 5/15
1/1 _____ 0s 84ms/step - accuracy: 0.9000 - loss: 1.6
Epoch 6/15
1/1 _____ 0s 83ms/step - accuracy: 0.6000 - loss: 1.6
Epoch 7/15
1/1 _____ 0s 83ms/step - accuracy: 0.7000 - loss: 1.6
Epoch 8/15
1/1 _____ 0s 90ms/step - accuracy: 0.8000 - loss: 0.9
Epoch 9/15
1/1 _____ 0s 85ms/step - accuracy: 0.8000 - loss: 0.9
Epoch 10/15
1/1 _____ 0s 83ms/step - accuracy: 1.0000 - loss: 0.9
Epoch 11/15
1/1 _____ 0s 83ms/step - accuracy: 1.0000 - loss: 0.8
Epoch 12/15
1/1 _____ 0s 92ms/step - accuracy: 0.9000 - loss: 0.7
Epoch 13/15
1/1 _____ 0s 87ms/step - accuracy: 1.0000 - loss: 0.7
Epoch 14/15
1/1 _____ 0s 86ms/step - accuracy: 1.0000 - loss: 0.7
Epoch 15/15
1/1 _____ 0s 81ms/step - accuracy: 1.0000 - loss: 0.6
1/1 _____ 0s 94ms/step

=== EXP LOG ENTRY ===
ID: EXP-04 | LR: 0.005 | Hidden: 128 | Dropout: 0.4 | Final Val Loss

```

- `if CONFIG["optimizer_type"] == "Adam":`: Checks if the value assigned to `"optimizer_type"` in the `CONFIG` dictionary is equal to `"Adam"`.
- `opt = Adam(learning_rate=CONFIG["learning_rate"])`: Instantiates the `Adam` optimizer set to the learning rate defined in `CONFIG` (`0.005`).
- `else:`: Serves as the fallback condition if `"optimizer_type"` is configured to anything other than `"Adam"`.
- `opt = SGD(learning_rate=CONFIG["learning_rate"])`: Instantiates the Stochastic Gradient Descent (`SGD`) optimizer using the specified learning rate.
- `model.compile(...)`: Configures the model for training by specifying the loss function, optimizer, and performance metrics to monitor:
 - `loss='sparse_categorical_crossentropy'`: Computes cross-entropy loss between multi-class integer targets and probability

outputs (avoiding the need for one-hot encoded labels).

- `optimizer=opt`: Assigns the instantiated optimizer (`Adam` or `SGD`).
- `metrics=['accuracy']`: Instructs TensorFlow to monitor classification accuracy during training and evaluation.
- `model.summary()`: Outputs a structured summary of the model architecture, displaying layer names, output dimensions, and total trainable vs. non-trainable parameter counts.
- `print(f"--- Starting Training Run: {CONFIG['run_id']} ---")`: Prints a formatted banner to the console displaying the current experiment identifier (`EXP-01-KERAS-BASELINE`).
- `history = model.fit(...)`: Executes the model training loop, logging loss and metric values per epoch into a `History` object:
 - `X_train, y_train`: Uses training features and target labels to compute gradients and update weights.
 - `validation_data=(X_val, y_val)`: Evaluates model performance on unseen validation data at the end of each epoch.
 - `epochs=CONFIG["epochs"]`: Sets training to run for 15 full passes over the dataset.
 - `batch_size=CONFIG["batch_size"]`: Processes samples in batches of 32 before updating model weights.
 - `verbose=1`: Enables an animated progress bar with real-time metric readouts per epoch.
- `val_preds_probs = model.predict(X_val)`: Runs forward passes on the validation set (`X_val`) to generate raw predicted class probabilities for each sample.
- `val_preds = np.argmax(val_preds_probs, axis=1)`: Identifies the class index with the highest predicted probability for each row along `axis=1`, converting soft probability distributions into hard predicted class labels.
- `val_f1 = f1_score(y_val, val_preds, average='weighted')`: Computes the weighted F1-score comparing ground truth validation labels (`y_val`) against predictions (`val_preds`), weighting each class score by its support to account for class imbalance.
- `val_loss = history.history['val_loss'][-1]`: Retrieves the validation loss value recorded during the final training epoch from the

(history) log.

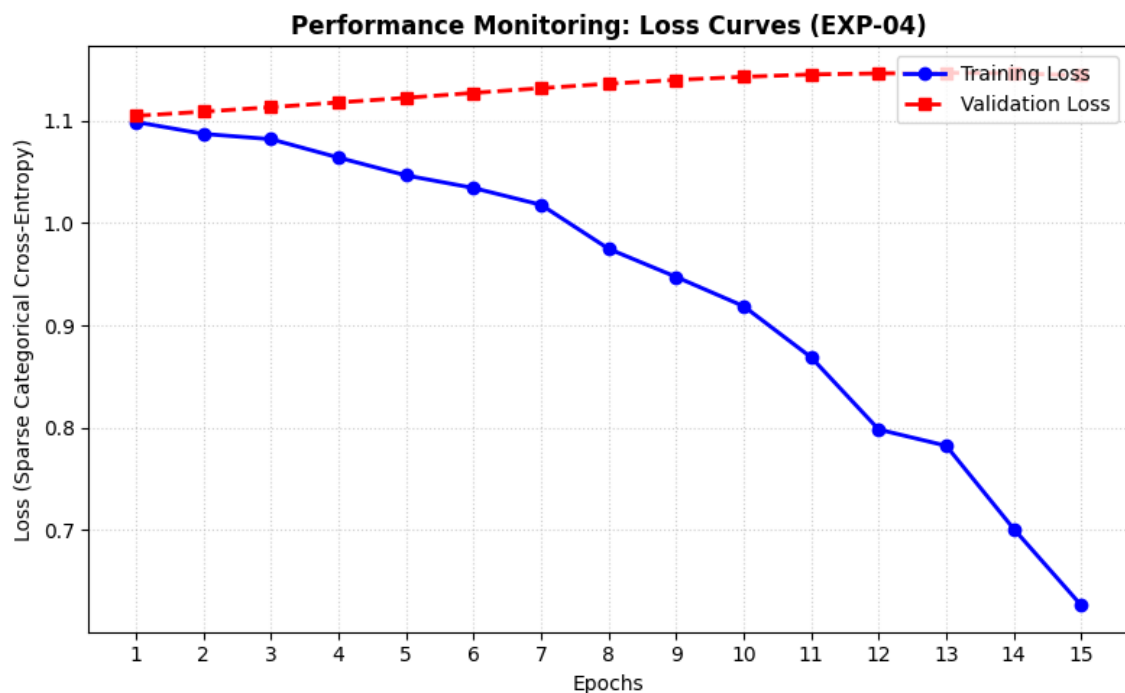
- `print("\n=== EXP LOG ENTRY ===")`: Prints a clean header for the experiment summary output block.
- `print(f"ID: ...")`: Formats and prints a single-line summary string containing experiment hyperparameters alongside final validation performance metrics (Loss and F1-score rounded to 4 decimal places).

```
train_losses = history.history['loss']
val_losses = history.history['val_loss']
epochs_range = range(1, CONFIG["epochs"] + 1)

plt.figure(figsize=(8, 5))
plt.plot(epochs_range, train_losses, label='Training Loss', color='blue')
plt.plot(epochs_range, val_losses, label='Validation Loss', color='red')

plt.title(f'Performance Monitoring: Loss Curves ({CONFIG["run_id"]})')
plt.xlabel('Epochs', fontsize=10)
plt.ylabel('Loss (Sparse Categorical Cross-Entropy)', fontsize=10)
plt.xticks(epochs_range)
plt.grid(True, linestyle=':', alpha=0.6)
plt.legend(loc='upper right')
plt.tight_layout()

plt.show()
```



- `train_losses = history.history['loss']`: Extracts the list of training loss values recorded at the end of each epoch from the `history` object's log dictionary.
- `val_losses = history.history['val_loss']`: Extracts the list of validation loss values recorded at the end of each epoch from the `history` object's log dictionary.
- `epochs_range = range(1, CONFIG["epochs"] + 1)`: Creates a sequence of integers from `1` up to the total number of epochs (`15`) to represent epoch counts on the x-axis.
- `plt.figure(figsize=(8, 5))`: Initializes a new plot canvas with a specified width of 8 inches and a height of 5 inches.
- `plt.plot(epochs_range, train_losses, label='Training Loss', color='blue', linewidth=2, marker='o')`: Plots the training loss curve as a solid blue line (width 2) with circular data point markers (`'o'`) and sets its legend label.
- `plt.plot(epochs_range, val_losses, label='Validation Loss', color='red', linewidth=2, linestyle='--', marker='s')`: Plots the validation loss curve as a dashed red line (`'--'`) of width 2 with square data point markers (`'s'`) and sets its legend label.
- `plt.title(f'Performance Monitoring: Loss Curves ({CONFIG["run_id"]})', fontsize=12, fontweight='bold')`: Sets the main plot title including the dynamic experiment run ID (`EXP-01-KERAS-BASELINE`) in bold text at font size 12.
- `plt.xlabel('Epochs', fontsize=10)`: Labels the horizontal x-axis as `'Epochs'`.
- `plt.ylabel('Loss (Sparse Categorical Cross-Entropy)', fontsize=10)`: Labels the vertical y-axis to indicate the loss metric being plotted.
- `plt.xticks(epochs_range)`: Ensures integer tick marks are explicitly displayed for every epoch along the x-axis (1 through 15).
- `plt.grid(True, linestyle=':', alpha=0.6)`: Overlays a dotted background grid (`':'`) at 60% opacity (`alpha=0.6`) to make reading specific values easier.
- `plt.legend(loc='upper right')`: Displays the color legend box

positioned in the top-right corner of the chart area.

- `plt.tight_layout()`: Automatically adjusts subplot margins to prevent titles, tick labels, or axes from clipping or overlapping.
 - `plt.show()`: Renders and displays the finalized figure output.
-
-